

Technology Stack

Date	15 March 2026
Team ID	TM-OPTICROP-04
Project Name	Smart Agricultural Production Optimization Engine (OptiCrop)
Maximum Marks	2 Marks

Define Your Technology Stack:

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs that you will use to build your solution. A well-chosen technology stack ensures that your application is scalable, maintainable, and aligned with your project requirements.

Fill out the table below to detail the specific technologies chosen for each component of your architecture and provide a brief justification for your choice.

Technology Stack Details

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side (e.g., React, Angular, HTML/ CSS)	React.js & Tailwind CSS	Chosen to build a highly responsive, component-based single-page application dashboard. Tailwind CSS ensures rapidly customizable, low-overhead UI layouts tailored for smooth monitoring views across both laptop display formats and mobile interfaces.
2	Backend / Server-Side (e.g., Node.js, Python/ Django, Java)	Python (FastAPI / Django)	Leveraged for its native alignment with high-performance ML pipelines and standard data processing layers. FastAPI handles concurrent, high-throughput asynchronous telemetry streaming routes with low latency, while managing background data ingestion scripts.
3	Database / Data Storage (e.g., MySQL, MongoDB, PostgreSQL)	PostgreSQL & TimescaleDB	PostgreSQL offers strong relational mapping and robust data consistency for user profile states, combined with the TimescaleDB extension to process heavy, time-series telemetry matrix records generated continuously by active field sensors.

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
4	Cloud / Hosting / Deployment <i>(e.g., AWS, Azure, Heroku, Docker)</i>	AWS (EC2, RDS) & Docker	Docker isolates localized microservices to ensure identical cross-environment runtimes. Container pipelines deploy smoothly into Amazon EC2 instances with managed AWS RDS setups to scale cloud resource allocation automatically as data volumes expand.
5	Version Control & CI/CD <i>(e.g., GitHub, GitLab, Jenkins)</i>	GitHub & GitHub Actions	Facilitates central version control and multi-developer workspace synchronization. Continuous Integration (CI) automated pipelines run code styling verifications, integration tests, and build checks upon every repository push to prevent broken updates.
6	Third-Party APIs / Other Tools <i>(e.g., Stripe, Twilio, Google Maps)</i>	OpenWeatherMap API & TensorFlow	Integrates live external micro-climatic forecasting matrices directly into the farm analysis pipelines. Embedded TensorFlow instances process image data features locally to support automatic object classification and condition detection updates.